GenAI Used: Gemini (accessed on KU Linux machine using Firefox)

Prompt Given:

{Assignment6.cpp included as attachment}

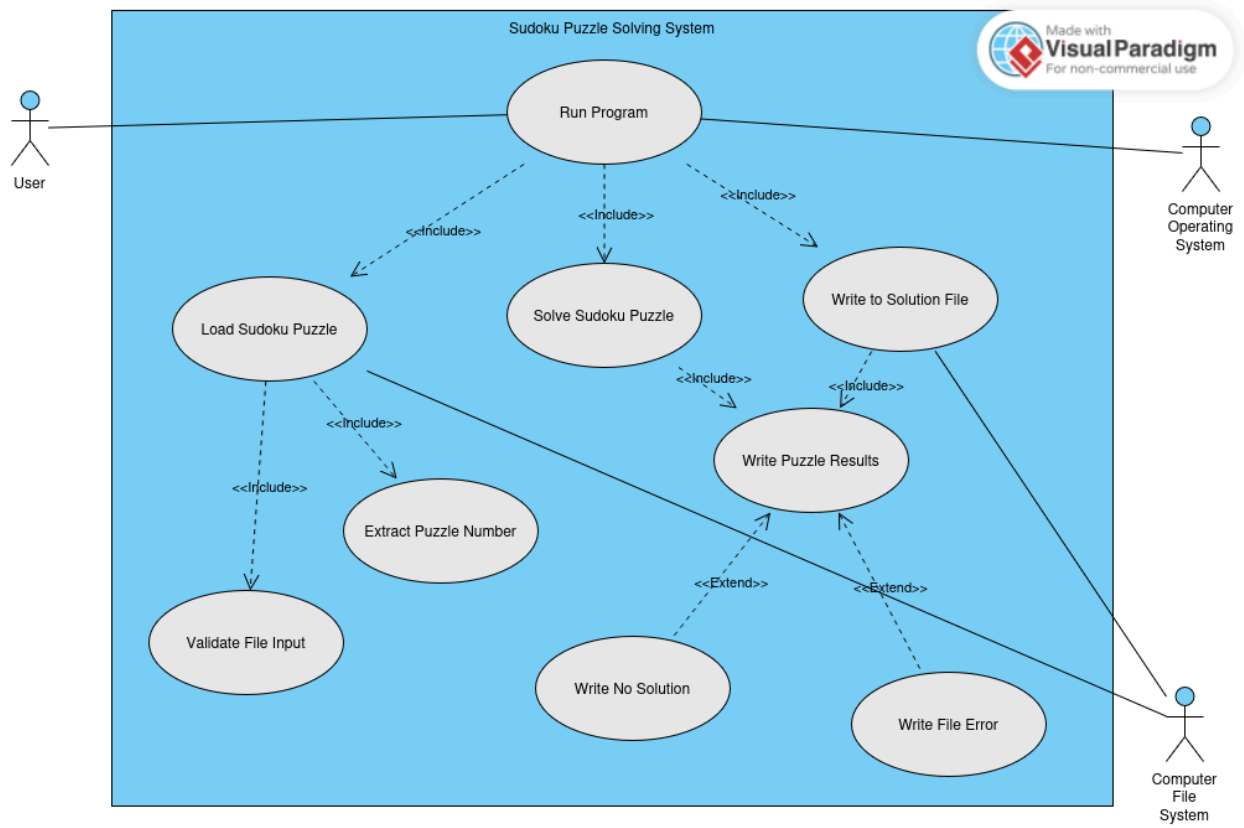
Attached is a C++ program that solves sudoku puzzles from text files. When the program solves a puzzle, it writes the results of all the puzzles to one text file.

Given the program, create a UML Use Case Diagram using these steps for each use case involved in the overall program:

a. Identify/discover a use case
b. Provide a brief description for the use case
c. Provide a step-by-step flow (outline) of the events for the use case
d. Provide a detailed description for each step of the use case outline

Use cases that already exist include: Run Program, Solve Sudoku Puzzle, Display Puzzle Results, Display No Solution, Load Sudoku Puzzle, Validate Puzzle Input, Display File Error, and Write Solution File. Define any additional use cases that describe the process of outputting the puzzle solutions to a file using the description steps mentioned above.

Graphic made with Visual Paradigm using GenAI output and previous UML Use Case diagram as guide:



➜ Use Case: Run Program
- Brief Description:
  - Starts program execution
- Outline:
  - Navigate to folder containing executable and puzzle files
  - Run program
- Detailed Description:
  - The user uses the terminal or the file navigator to navigate to the folder containing puzzle files 1- 5 and the program executable
  - The user runs the program by entering the corresponding command into the terminal
  - The program solves the puzzle files and outputs the result into a file

➜ Use Case: Load Sudoku Puzzle
- Brief Description:
  - The program loads the sudoku puzzle from the current puzzle file to prepare it for solving.
- Outline:
  - Open puzzle file
  - Read each cell value by row and then by cell
  - If input is valid, puzzle is parsed and ready
- Detailed Description:
  - The program opens the puzzle file and reads it line by line
  - Each line is checked to confirm its cells are correctly formatted (0-9 or _)
  - The input is validated to check cell contents and dimensions
  - If the puzzle is valid, the program stores its contents into a 2D vector

➜ Use Case: Validate Puzzle Input
- Brief Description:
  - The program confirms that the input given to create a grid follows standard sudoku rules.
- Outline:
  - Check that there are 81 cells
  - Check that the input is 9x9
  - Check that at least 17 cells are filled
  - Check that cells only contain either a digit or an underscore
  - Check that there are no duplicate numbers
- Detailed Description:
  - The program counts each cell it checks and compares the sum to a total of 81.
  - The program checks that there are 9 rows, and that each row is 9 cells long.
  - The program checks that at least 17 cells have a digit character to ensure that the puzzle is solvable.

        - The program checks that the cells only contain either a digit character or an underscore. Any other characters are not accepted and invalid.
        - The program checks that a number does not appear twice in a square, row, or column.
        - If the puzzle input fails any of these checks, the validity of the grid is false.
- ➜ Use Case: Write File Error
    - Brief Description:
        - If the program cannot open the file containing the puzzle, or the puzzle input is invalid, write the error to the solution file.
    - Outline:
        - Detect file open failure or invalid puzzle input
        - Write the error message to the outstream
        - End the parsing process for the current file
    - Detailed Description:
        - The program cannot access or open the file, or the input is deemed invalid by the validation process.
        - The program writes the reason to the outstream that will be written to the output file.
        - The program ends the parsing process for the current file and moves to process the next file.
- ➜ Use Case: Solve Sudoku Puzzle
    - Brief Description:
        - Solve the sudoku puzzle using a recursive DFS (with backtracking) object oriented approach.
    - Outline:
        - Find the next empty cell.
        - Place the current candidate value, and if it is valid, move to the next empty cell.
        - If a conflict is found, backtrack to the cell and place the next candidate value.
        - If no empty cells are left, mark the puzzle as solved.
    - Detailed Description:
        - The program searches for the next cell that is marked as empty with an _ character.
        - The program places the first candidate value in the cell after checking that it does not already exist in the current, row, column, or subgrid.
        - If the candidate is valid, the program will recurse to the next blank cell.
        - If the candidate is invalid, the program will backtrack to the previous cell, place the next candidate value, and then return to the current cell to try the next appropriate candidate value.
- ➜ Use Case: Write Puzzle Results
    - Brief Description:
        - The puzzle solution or solutions are written to the file output.
    - Outline:

        - Solve puzzle
        - Save solutions
        - Write to output stream
    - Detailed Description:
        - After solving the puzzle, the program prints the current puzzle, the solutions, and the number of solutions to the file outstream.
        - The solving function then continues to the next puzzle and its concurrent results.
        - After all the puzzles are solved, the main function writes the complete outstream to the solutions file.
- ➜ Use Case: Write No Solution
    - Brief Description:
        - If no solution is found, the corresponding message is written to the file output.
    - Outline:
        - Attempt to solve puzzle
        - Report no solution
        - Write result to output filestream
    - Detailed Description:
        - After attempting to solve the puzzle, if the program cannot find any solutions, it writes the result to the output stream.
        - The program then moves to the next puzzle.
- ➜ Use Case: Write to Solution File
    - Brief Description:
        - After all the puzzles are solved, all the results are written to the solutions file.
    - Outline:
        - Return output stream containing solutions
        - Create or open solutions file
        - Write output stream to file
        - Close solutions file
    - Detailed Description:
        - After solving all the puzzles, the program returns the complete outstream to the main function.
        - The main function creates a solutions file if it does not exist yet. Otherwise, it opens the solutions file.
        - The main function writes the outstream containing the puzzle results to the solutions file.
        - The main function closes the solutions file and ends the program.
- ➜ Use Case: Extract Puzzle Number (Additional Use Case)
    - Brief Description:
        - The current puzzle number is extracted from the current puzzle file's name.
    - Outline:

    - Receive the filename
    - Locate the number
    - Extract from file name
- Detailed Description:
    - The program receives the puzzle filename from the main program.
    - The program locates where the number starts and ends by finding the first digit character and the dot character which indicates the start of the filename extension and the end of the filename.
    - Using the indices of these characters, the program creates a substring of only the number of the puzzle file to use later in the solution output.